Robot Predictive Maintenance — Season 2026

Project Briefing for LLM-Assisted Development

Compiled from the Kaggle competition pages and the data package supplied to participants

Document compiled 2026-05-28

Contents

1 Document purpose

This document is a self-contained briefing pack for the Kaggle competition *Robot predictive maintenance — Season 2026*. It is intended to be fed to a large language model that will help the author build a solution. It consolidates, in one place:

- the competition's stated overview, rules and evaluation;
- the description of the physical system and the failure mode;
- the exact structure of the provided dataset (training and testing);
- the format of the submission file;
- the full source of the provided `utility.py` helper module;
- a best-effort transcription of the failure-injection routine that synthesises the labels in the training data.

The document is descriptive: it does not recommend a modelling approach. It is written so that an LLM reading only this file can answer questions about the problem, the data layout and the helper API without needing access to the original Kaggle pages.

Source material:

- Kaggle competition pages (Overview, Description, Evaluation, Data, Citation), captured 2026-05-28.
- Files supplied in the download `robot-predictive-maintenance-season-2026/`: `utility.py`, `sample_submission.csv`, `training_data/`, `testing_data/`.

2 Competition snapshot

Title	Robot predictive maintenance — Season 2026
Platform	Kaggle (Community Prediction Competition)
Host	Zhiguo Zeng
Organising body	Chair Risk and Resilience of Complex Systems
Funding ack.	French research support; contract `ANR-22-CE-10-0004-OT1`
License	CC BY-NC-SA 4.0
Data package	190 files, 9.92 MB (CSV, XLSX, PY)
Entrants	60 (at time of capture)
Citation	Zhiguo Zeng. *Robot predictive maintenance — Season 2026.* https://kaggle.com/competitions/robot-predictive-maintenance-season-2026, 2026. Kaggle.

3 The physical system

The competition is based on an educational robot ArmPi FPV from Hiwonder. The robot has the following characteristics:

- Six degrees of freedom, driven by six serial-bus servo motors.

- Controlled by a Raspberry Pi single-board computer and a Roli Metallic controller.
- Each servo motor reports its **position**, **temperature** and **voltage** when polled.

A condition-monitoring program is developed on the Raspberry Pi and communicates with MATLAB to poll the position, temperature and voltage of all six motors in real time. The nominal sampling period is approximately **0.1 s** (i.e. $\sim$10 Hz). Acquisitions used in the dataset last on the order of 30 s up to several minutes per test sequence.

4 Failure mode

The fault studied in the competition is an **abnormal temperature rise on a motor**. Per the competition description, this failure mode can occur for many reasons in practice and is hard to predict from raw signals alone.

In the dataset, failures are **simulated synthetically**: a non-linear temperature rise is injected into the temperature channel of the affected motor(s) over a contiguous span of the sequence, and the corresponding `label` column is set to `1` over that span. Outside the injected span, the signal is unchanged and the label is `0`.

The injection routine itself is shown in Section 9. Two consequences for modelling that follow directly from the synthetic nature of the labels:

- The fault signature is whatever shape the injection function produces; it is not in-the-wild fault data.
- Outside the injected span, the affected motor's temperature channel is identical to a healthy run; the discriminative signal is concentrated inside the injected span.

5 Task definition

The objective is to predict the health state of each of the six motors at each time step of each test sequence:

- `0` — working normally;
- `1` — abnormal temperature rise.

This yields **six independent binary classification problems** (one per motor), evaluated jointly via the mean of their F1 scores (Section 7).

6 Dataset description

6.1 Top-level layout

The data package supplied to participants has the following top-level layout:

```
robot-predictive-maintenance-season-2026/
    utility.py
    sample_submission.csv
```

```
training_data/
    training_data/
        Test conditions.xlsx
        <test_id_1>/
            data_motor_1.csv
            data_motor_2.csv
            data_motor_3.csv
            data_motor_4.csv
            data_motor_5.csv
            data_motor_6.csv
        <test_id_2>/
            ...
testing_data/
    testing_data/
        Test conditions.xlsx
        <test_id>/
            data_motor_1.csv
            ...
```

Each "test_id" is an underscore-separated timestamp of the form `YYYYMMDD_xxxxxx`. Inside each test-id folder, six CSV files contain the time series of the six motors collected during that test.

6.2 Per-motor CSV schema

Every `data_motor_k.csv` file has the same five columns:

`time`	Wall-clock timestamp in seconds (large absolute values; effectively a counter — see note below).
`position`	Servo position reported by the motor (integer units).
`temperature`	Motor temperature (integer, °C).
`voltage`	Motor supply voltage (integer, mV-scale).
`label`	Ground-truth label: `0` = normal, `1` = abnormal temperature rise. (Present in training data; intentionally not provided for testing data.)

Note on the time column. The `time` values are large absolute seconds. The helper function `read_all_csvs_one_test` subtracts the first row to obtain a relative time, and falls back to a synthetic sequence 0, 0.1, 0.2, ... if subtraction fails. A sample of the raw values from one of the training motor CSVs:

```
time,position,temperature,voltage,label
76522.025432825,86,42,7223,0
76522.125464200,86,42,7214,0
76522.225431919,86,42,7137,0
76522.325432062,86,42,7135,0
...
```

6.3 `Test conditions.xlsx` (training)

The training-side spreadsheet lists the 23 training sequences and, for each one, the textual description of the test and a per-motor failure flag (NaN if no failure was injected on that motor, `1.0` if a failure was injected). Table 1 reproduces the contents.

Table 1: Training sequences — contents of `Test conditions.xlsx`. "Failed motors" lists the motors for which the corresponding `Motor_k_failure` cell is `1.0`; "–" means no failure flag was set.

Test id	Description	Failed motors
20240105_164214	Not moving	–
20240105_165300	Pick up and place	–
20240105_165972	Not moving	–
20240320_152031	Not moving	–
20240320_153841	Move motor 6 → motor 1 sequentially	–
20240320_155664	Not moving	–
20240321_122650	Pick up and place	–
20240325_135213	Not moving	–
20240325_152902	Pick up and place	–
20240325_155003	Pick up and place	1, 2, 3, 4, 5, 6
20240425_093699	Not moving	6
20240425_094425	Not moving	6
20240426_140055	Not moving	1, 2, 3, 4, 5, 6
20240426_141190	Pick up and place	–
20240426_141532	Moving one motor	–
20240426_141602	Moving one motor	–
20240426_141726	Moving one motor	–
20240426_141938	Moving one motor	–
20240426_141980	Moving one motor	–
20240503_163963	Pick up and place	6
20240503_164435	Turning motor 6	–
20240503_164675	Turning motor 6	6
20240503_165189	Turning motor 6	6

Observations on the training distribution.

- 23 training sequences in total.
- 16 sequences contain no injected failure on any motor; 7 sequences contain at least one injected failure.
- Of the 7 faulty sequences, 5 are motor-6 only, and 2 inject failures on all six motors simultaneously.
- Motor 6 is therefore by far the most frequently faulted motor in the training data; motors 1–5 appear as faulty only in the two "all-motors-faulty" sequences.
- The activity descriptions cover: *Not moving*, *Pick up and place*, *Moving one motor*, *Turning motor 6*, and *Move motor 6 → motor 1 sequentially*.

6.4 `Test conditions.xlsx` (testing)

The testing-side spreadsheet has only two columns: `Test id` and `Description`. No failure information is provided; the participant must predict it. Table 2 reproduces the contents and adds the row count of the corresponding submission slice (see Section 6.5).

Table 2: Testing sequences. Row counts are the number of rows the corresponding `test_condition` occupies in `sample_submission.csv`.

Test id	Description	Rows in submission
20240527_094865	Transfer goods	2 423
20240527_100759	Transfer goods	3 187
20240527_101627	Transfer goods	2 192
20240527_102436	Not moving	1 548
20240527_102919	Not moving	700
20240527_103311	Not moving	1 239
20240527_103690	Moving one motor	745
20240527_104247	Moving one motor	2 123
Total		**14 157**

The test descriptions include one activity not present in the training set (*Transfer goods*) and three that are (*Not moving*, *Moving one motor*, and implicitly *Pick up and place* via *Transfer goods*).

6.5 Submission file: `sample_submission.csv`

`sample_submission.csv` has **14 158 lines** (header + 14 157 data rows) and the following columns:

```
idx, data_motor_1_label, data_motor_2_label, data_motor_3_label,
data_motor_4_label, data_motor_5_label, data_motor_6_label, test_condition
```

- `idx` is a contiguous 0-based row index running from 0 to 14 156.
- All label columns are initialised to `-1` as a placeholder; the submission must replace every `-1` with `0` or `1`.
- `test_condition` is the test-id of the testing sequence the row belongs to. The eight test ids appear in contiguous blocks of the sizes given in Table 2.

The first lines of the file are:

```
idx,data_motor_1_label,data_motor_2_label,data_motor_3_label,
data_motor_4_label,data_motor_5_label,data_motor_6_label,test_condition
0,-1,-1,-1,-1,-1,-1,20240527_094865
1,-1,-1,-1,-1,-1,-1,20240527_094865
2,-1,-1,-1,-1,-1,-1,20240527_094865
...
```

7 Evaluation

The competition is scored on the **mean F1 score across the six motors**. For each motor $k \in \{1, \ldots, 6\}$, the F1 score is computed between the ground-truth column `data_motor_`k`_label` in the held-out solution file and the corresponding column in the participant's submission, treating the time-step rows as independent samples. The six per-motor F1 scores are then averaged.

Zero-division behaviour is explicit:

- If both `y_true` and `y_pred` contain no positive labels (i.e. both sums are zero) for a motor, F1 is set to `1.0` for that motor.
- Otherwise, the standard scikit-learn F1 with `zero_division=0` is used.

The exact scoring code published with the competition is reproduced below.

Per-motor F1 helper

```python
def cal_classification_perf(y_true, y_pred):
    ''' ### Description
    This function calculates the classification performance: Accuracy,
    Precision, Recall and F1 score.
    It considers different scenarios when divide by zero could occur for
    Precision, Recall and F1 score calculation.

    ### Parameters:
    - y_true: The true labels.
    - y_pred: The predicted labels.

    ### Return:
    - f1: The F1 score.
    '''
    # Only when y_pred contains no zeros, and y_true contains no zeros,
    # set precision to be 1 when divide by zero occurs.
    if sum(y_true) == 0 and sum(y_pred) == 0:
        f1 = f1_score(y_true, y_pred, zero_division=1)
    else:
        f1 = f1_score(y_true, y_pred, zero_division=0)

    return f1
```

Top-level scoring function

```python
import pandas as pd
import pandas.api.types
from sklearn.metrics import f1_score


class ParticipantVisibleError(Exception):
    # If you want an error message to be shown to participants, you must
    # raise the error as a ParticipantVisibleError.
    # All other errors will only be shown to the competition host. This
    # helps prevent unintentional leakage of solution data.
    pass


def score(solution: pd.DataFrame,
          submission: pd.DataFrame,
          row_id_column_name: str = 'idx') -> float:
    '''
    >>> import pandas as pd
    >>> num_rows = 5
    >>> data = {'idx': list(range(num_rows))}
    >>> for i in range(1, 7):
    ... column_name = f'data_motor_{i}_label'
    ... data[column_name] = [1] * num_rows
    >>> solution = pd.DataFrame(data)
    >>> submission = solution
    >>> score(solution.copy(), submission.copy())
    1.0
    '''
    del solution[row_id_column_name]
    del submission[row_id_column_name]

    results = []
    for i in range(6):
        col_name = f'data_motor_{i+1}_label'
```

```
        y_true = solution[col_name].values
        y_pred = submission[col_name].values
        result = cal_classification_perf(y_true, y_pred)
        results.append(result)

    score_result = sum(results) / len(results)
    return score_result
```

Implications.

- The score is invariant to the ordering of rows within a test condition only insofar as the scoring code drops `idx` but keeps row order from the dataframes; submissions should preserve the row order of `sample_submission.csv`.
- The `test_condition` column is not used by the scoring code (only the six label columns and the dropped `idx` are consumed), but it must still be present for the submission to match the expected schema.

8 Provided helper module: `utility.py`

A single Python module, `utility.py`, is supplied with the competition. It contains data-loading helpers, a sliding-window feature builder, a sequence-aware cross-validation routine, classification-metric utilities, a regression-residual-based fault detector class (`FaultDetectReg`), plotting helpers, and a self-contained `__main__` demo using a `StandardScaler`+`LinearRegression` pipeline.

A function-by-function API reference is given in Appendix Appendix A. The full verbatim source of the module is reproduced in Appendix Appendix D.

The following subsections summarise the key concepts introduced by the helper, because they are referenced throughout the rest of the briefing.

8.1 Sequence as the unit of data

The whole helper is organised around the concept of a *sequence* (also called a *test condition*): one row of `Test conditions.xlsx`, one sub-folder on disk, one block of contiguous rows in `sample_submission.csv`. Sequences are identified by the string column `test_condition` that is added when CSVs are loaded.

The cross-validation routines split *by sequence*, not by row, to avoid leakage between train and validation folds.

8.2 Wide dataframe layout

`read_all_csvs_one_test` merges the six per-motor CSVs of a single test into one wide dataframe by prefixing each motor file's columns with `data_motor_`*k*`_`. The columns of the resulting wide frame are therefore:

```
time,
data_motor_1_position, data_motor_1_temperature,
data_motor_1_voltage,  data_motor_1_label,
data_motor_2_position, ... data_motor_6_label,
test_condition
```

`read_all_test_data_from_path` iterates this over every test-id folder under a base directory and concatenates the results. An optional `pre_processing` callback is applied to each per-motor frame before merging (the demo uses a `remove_outliers` callback — see Section 8.5).

8.3 Sliding-window features

`prepare_sliding_window` and `concatenate_features` turn the wide time-series frame into a flat feature matrix by sliding a window of size `window_size` across each sequence and concatenating `floor(window_size / sample_step)` time steps' worth of features into a single row. For regression-style models, lagged values of the response variable up to `prediction_lead_time` steps in the past can also be appended as features.

Important boundary behaviours:

- The first `window_size - 1` rows of a sequence are padded by repeating the first row, so the output length equals the input length when the padding branch is used.
- Cross-validation and prediction in `FaultDetectReg` call the feature builder *per sequence*, never across sequence boundaries.

8.4 Regression-residual fault detector

`FaultDetectReg` predicts a motor's temperature from the wide-frame features using a wrapped regression model, computes the absolute residual between the prediction and the measurement, and emits label `1` when the residual exceeds an adaptive threshold. Key behaviours, taken verbatim from the source:

- **Adaptive threshold.** After each new normal sample, the running `mean(residual_norm) + 6 * std(residual_norm)` is computed; the working threshold is raised to that value if it is between the current threshold and the hard cap `1.5`.
- **Closed-loop correction.** When the detector flags a fault and the residual is at most `abnormal_limit`, the predicted temperature is substituted for the measurement in subsequent windowed features, to prevent error accumulation through the lagged response inputs.
- **Truncation versus padding.** By default the predicted label vector has length `N - window_size + 1`; passing `complement_truncation=True` pads the leading positions by repeating the first prediction so the output length equals the input length. This matters for submission alignment.

8.5 Demo in `__main__`

When `utility.py` is executed directly, the bottom of the file runs an example pipeline. It:

1. Reads all training data from an absolute path that points at the original author's machine (`C:/Users/Zhiguo/OneDrive - CentraleSupelec/...`); this path is not portable and must be replaced before running the demo elsewhere.
2. Applies a custom `remove_outliers` callback that clips temperature to $[0, 100]\,^{\circ}\mathrm{C}$, voltage to $[6000, 9000]$, and position to $[0, 1000]$, forward-fills invalid entries, then subtracts the first valid value of each channel.

3. Restricts training to a hard-coded list of 15 "normal" test ids and trains a `StandardScaler` → `LinearRegression` pipeline as the regression backbone.

4. Uses these hyperparameters throughout: `window_size=10`, `sample_step=1`, `prediction_lead_time=1`, `threshold=0.8`, `abnormal_limit=3`, and motor index 6.

5. Constructs a `FaultDetectReg` around the pipeline and runs prediction on a hard-coded subset of two test sequences (`20240325_155003` and `20240425_093699`).

The demo is illustrative, not part of the evaluation. The exact list of "normal" test ids hard-coded into the demo is:

```
20240105_164214, 20240105_165300, 20240105_165972,
20240320_152031, 20240320_153841, 20240320_155664,
20240321_122650, 20240325_135213, 20240426_141190,
20240426_141532, 20240426_141602, 20240426_141726,
20240426_141938, 20240426_141980, 20240503_164435
```

9 Failure-injection routine (best-effort transcription)

The Kaggle Description page includes a screenshot of the routine used to synthesise the failure signature in the training data. The original screenshot is at low resolution; the transcription below is best-effort and reproduces the routine's logic but may differ in cosmetic details (variable names, exact indices) from the upstream source. The routine is written in MATLAB style.

```
function temperature = inject_failure(temperature, label)
    % Get the response above the temperature
    tmp_temp = temperature(label == 1);

    % Length of the response
    n_seq = length(tmp_temp);

    % Generate a random failure temperature rise.
    % We assume the temperature decrease is 1 time slower
    % than the temperature rise.
    n_rise = floor(n_seq / 4);

    if ~isempty(tmp_temp)
        % Get the starting and ending temperatures.
        temp_start = tmp_temp(1);
        temp_end = tmp_temp(end);

        % Generate a random highest temperature.
        temp_high = max(temp_start, temp_end) + randi([2, 50]);

        % Generate the temperature rise.
        temp_size_rise = floor(n_rise / (temp_high - temp_start));
        for i = 1:(temp_high - temp_start)
            tmp_temp(1 + (i-1)*temp_size_rise : i*temp_size_rise) = ...
                temp_start + i;
        end
        tmp_temp(i*temp_size_rise + 1 : end) = temp_high;

        % Generate the temperature decrease.
        n_down = n_seq - n_rise;
        temp_size_down = floor(n_down / (temp_high - temp_end));
        for i = 1:(temp_high - temp_end)
            tmp_temp(n_rise + 1 + (i-1)*temp_size_down : ...
                    n_rise + i*temp_size_down) = temp_high - i;
```

```
        end
        tmp_temp(n_rise + i*temp_size_down + 1 : end) = temp_end;
    end

    temperature(label == 1) = tmp_temp;
end
```

The qualitative behaviour the routine encodes, and which can be relied upon regardless of the cosmetic details above:

- The injection acts only on the contiguous span where `label == 1`.
- Inside that span, the signal is replaced by a piecewise-linear triangular waveform: it ramps up from the starting temperature to a randomly chosen peak (`randi([2,50])` degrees above the baseline), then ramps back down to the ending temperature.
- The rise portion occupies roughly the first quarter of the span and the decay portion occupies the remaining three quarters; rises are therefore steeper than decays.
- The peak amplitude and the exact span boundaries are randomised, so each injected sequence has a different magnitude and a different triangle width.

10 Submission checklist

For an LLM generating a submission file, the following invariants must hold:

- File name: `submission.csv` (or whatever the Kaggle UI requests at upload).
- Column order, identical to `sample_submission.csv`:

  ```
  idx,
  data_motor_1_label, data_motor_2_label, data_motor_3_label,
  data_motor_4_label, data_motor_5_label, data_motor_6_label,
  test_condition
  ```

- Row count: exactly 14 157 data rows; `idx` contiguous from 0 to 14 156, in the same order as the sample.
- Each `test_condition` block must contain the row counts in Table 2, in the same order as the sample.
- Every `-1` placeholder in the six label columns must be replaced by `0` or `1`; leaving any `-1` will ruin the F1 score for the affected motor.
- Labels are integers, not floats; the scoring function uses `sklearn.metrics.f1_score` directly on the column values.

Appendix A. Function-by-function API reference for `utility.py`

The entries are presented as a description list rather than a table to keep function signatures and their explanations together on the page.

`FaultDetectReg (class)`
Regression-residual fault detector. Wraps a pre-trained regression model, predicts the response, thresholds the absolute residual, and adapts the threshold from observed normal residuals.

`__init__`
Stores the regression model and the configuration knobs: `threshold`, `abnormal_limit`, `window_size`, `sample_step`, `pred_lead_time`, and the `pre_trained` flag.

`fit`
If `pre_trained=False`, trains the regression model on the rows with `label == 0` (re-segmenting the sequence each time the label toggles), then runs prediction on the input.

`predict`
Predicts labels and response variable, sequence by sequence, using a sliding window. Substitutes predicted response for measured response when a fault is flagged and the residual is at most `abnormal_limit`. Optionally pads the output to match the input length (`complement_truncation`).

`predict_label_by_reg_base`
Inner per-window step: predicts response, computes absolute residual, emits label, updates the adaptive threshold using `mean + 6·std` of stored normal residuals (capped at `1.5`).

`run_cross_val`
Sequence-aware K-fold cross-validation. Splits by `test_condition`, optionally re-trains per fold, returns accuracy, precision, recall, F1 for each fold.

`extract_selected_feature`
Splits a wide dataframe into a feature dataframe and a response Series (label or temperature) for a given motor and model type (`'clf'` or `'reg'`). Always keeps the `test_condition` column on the feature side.

`extract_selected_feature_testing`
Same as above; intended for the test-time entry point.

`run_cv_one_motor`
Wraps feature extraction and `run_cross_val` for one motor; prints per-fold and mean $\pm$ std performance.

`read_all_test_data_from_path`
Walks every test-id sub-folder under a base directory, calls `read_all_csvs_one_test` on each, concatenates the results, and optionally plots the per-motor signals coloured by label. Reads `Test conditions.xlsx` for the descriptions.

`read_all_csvs_one_test`
Reads the six per-motor CSVs of one test, drops the original time column from each, prefixes the remaining columns with the file name, concatenates side-by-side, re-attaches a normalised time column (falling back to a synthetic 0.1 s grid), tags every row with `test_condition`, and drops rows that are NaN in any non-label column.

`concatenate_features`
Sub-routine that takes one window's worth of rows and a corresponding response slice, samples every `sample_step`-th row from the window, flattens the result into a single feature row, optionally appends lagged response values for regression mode, and appends the row to a growing feature list.

`prepare_sliding_window`
Iterates `concatenate_features` across every position of every sequence. Handles short prefixes by repeating the first row. Returns `(X, y)` as a dataframe and Series.

`run_cross_val (free function)`
Generic sequence-aware K-fold cross-validation that works for either a regression model (metrics: max error, RMSE, exceedance rate) or a classifier (metrics: accuracy, precision, recall, F1).

`cal_classification_perf`
Computes accuracy, precision, recall and F1 with explicit zero-division handling (precision/recall/F1 set to 1 when both vectors are all zeros).

`run_mdl`
Fits a model on the training matrix, returns the fitted model and the predictions on both training and testing matrices.

`show_reg_result* / show_clf_result*`
Matplotlib visualisations of regression residuals or classification predictions; print per-run performance summaries.

`remove_outliers (in __main__)`
Demo-only preprocessing callback that clips temperature to $[0, 100]$, voltage to $[6000, 9000]$, position to $[0, 1000]$, forward-fills the invalidated entries, and subtracts the first valid value of each channel.

Appendix B. File tree of the supplied data package

```
robot-predictive-maintenance-season-2026/
|-- sample_submission.csv         (14,158 lines = header + 14,157 rows)
|-- utility.py
|-- training_data/
|   `-- training_data/
|       |-- Test conditions.xlsx
|       |-- 20240105_164214/      Not moving
|       |-- 20240105_165300/      Pick up and place
|       |-- 20240105_165972/      Not moving
|       |-- 20240320_152031/      Not moving
|       |-- 20240320_153841/      Move motor 6 -> motor 1 sequentially
|       |-- 20240320_155664/      Not moving
|       |-- 20240321_122650/      Pick up and place
|       |-- 20240325_135213/      Not moving
|       |-- 20240325_152902/      Pick up and place
|       |-- 20240325_155003/      Pick up and place  [faults: 1-6]
|       |-- 20240425_093699/      Not moving         [fault: 6]
|       |-- 20240425_094425/      Not moving         [fault: 6]
|       |-- 20240426_140055/      Not moving         [faults: 1-6]
|       |-- 20240426_141190/      Pick up and place
|       |-- 20240426_141532/      Moving one motor
|       |-- 20240426_141602/      Moving one motor
|       |-- 20240426_141726/      Moving one motor
|       |-- 20240426_141938/      Moving one motor
|       |-- 20240426_141980/      Moving one motor
|       |-- 20240503_163963/      Pick up and place  [fault: 6]
|       |-- 20240503_164435/      Turning motor 6
|       |-- 20240503_164675/      Turning motor 6    [fault: 6]
|       `-- 20240503_165189/      Turning motor 6    [fault: 6]
`-- testing_data/
    `-- testing_data/
```

```
        |-- Test conditions.xlsx
        |-- 20240527_094865/      Transfer goods
        |-- 20240527_100759/      Transfer goods
        |-- 20240527_101627/      Transfer goods
        |-- 20240527_102436/      Not moving
        |-- 20240527_102919/      Not moving
        |-- 20240527_103311/      Not moving
        |-- 20240527_103690/      Moving one motor
        `-- 20240527_104247/      Moving one motor

Every test-id folder contains exactly six files:
    data_motor_1.csv ... data_motor_6.csv
with columns: time, position, temperature, voltage, label
('label' is omitted -- as a column it is absent or to be ignored -- in
 the testing data).
```

Appendix C. Glossary

Sequence / test condition
One acquisition of the robot, identified by a string of the form `YYYYMMDD_xxxxxx`. One folder on disk, one row of `Test conditions.xlsx`, one contiguous block in the submission file.

Wide frame
The dataframe produced by `read_all_csvs_one_test`: one row per time step, columns for each (motor, signal) pair, plus `time` and `test_condition`.

Window size
The number of consecutive time steps fed into the feature builder per prediction. The demo uses 10.

Sample step
Stride inside a window when sampling time steps to flatten into the feature row. The demo uses 1 (every step in the window is used).

Prediction lead time
Number of steps into the future the regression model predicts. The demo uses 1 (one-step-ahead).

Residual error
Absolute difference between the regression model's predicted temperature and the measured temperature. The basis for the binary label in `FaultDetectReg`.

Adaptive threshold
Working residual threshold that grows with the empirical `mean + 6·std` of stored normal residuals, capped at 1.5.

Truncation versus padding
With a window of size w, a length-N sequence yields $N - w + 1$ predictions; the helper can either return that shorter vector (*truncated*) or pad the leading positions by repeating the first prediction (*complemented*).

Appendix D. Full source of `utility.py`

The complete source of the module supplied with the competition is reproduced verbatim below. Line numbers are preserved.

```python
import matplotlib.pyplot as plt
from numpy.core.fromnumeric import shape
from sklearn.metrics import max_error, mean_squared_error, accuracy_score, precision_score, recall_score, f1_score
from sklearn.model_selection import KFold
from sklearn.preprocessing import StandardScaler
import numpy as np
import pandas as pd
from pathlib import Path
from pandas.plotting import scatter_matrix
import logging
import sys
import os
import matplotlib.pyplot as plt
import copy
from tqdm import tqdm


# We provide some supporting function for training a data-driven digital twin for predicting the temperature of motors.


class FaultDetectReg():
    ''' ### Description
    This is the class for fault detection based on regression models.

    ### Initialize
    Initialize the class with the following parameters:
    - reg_mdl: The pre-trained regression model.
    - pre_trained: If the provided reg_mdl is pretrained. Default is True.
    - threshold: threshold for the residual error. If the residual error is larger than the threshold, we consider it as a fault. Default is 1.
    - abnormal_limit: If the model predict a abnormal point and its residual error <= abnormal_limit, the predicted value will be used to replace
      the measured value in the next prediction. Default is 1.
    - window_size: Size of the sliding window. The previous window size points will be used to create a new feature.
    - sample_step: We take every sample_step points from the window_size. default is 1.
    - prediction_lead_time: The number of time steps to predict into the future. Only valid for regression model. Default is 1.

    ### Attributes
    - reg_mdl: The pre-trained regression model.
    - pre_trained: If the provided reg_mdl is pretrained. Default is True.
    - threshold: threshold for the residual error. If the residual error is larger than the threshold, we consider it as a fault. Default is 1.
    - abnormal_limit: If the model predict a abnormal point and its residual error <= abnormal_limit, the predicted value will be used to replace
      the measured value in the next prediction. Default is 1.
    - window_size: Size of the sliding window. The previous window size points will be used to create a new feature.
    - sample_step: We take every sample_step points from the window_size. default is 1.
    - prediction_lead_time: The number of time steps to predict into the future. Only valid for regression model. Default is 1.
    - residual_norm: The stored residual errors for all the normal samples in the current sequence.
      It is used to calculate the threshold adpatively.
    - threshold_int: The threshold value set by the initialization function.

    ### Methods
    - fit(): This method learns the regression model from the training data. If self.pre_trained is True, it will directly use the pre-trained
        ↪ model.
    - predict(): This method predicts the labels for the input data.
    - predict_label_by_reg_base(): This method is the base function for predicting the label, called from self.predict().
    - run_cross_val(): This method defines a cross validation scheme to test the performance of the model.

    '''
    def __init__(self, reg_mdl, pre_trained: bool=True, threshold: int=1, abnormal_limit: int=3, window_size: int = 1, sample_step: int = 1,
        ↪ pred_lead_time: int = 1):
        ''' ### Description
        Initialization function.
        '''
        self.reg_mdl = reg_mdl
        self.window_size = window_size
        self.sample_step = sample_step
        self.pred_lead_time = pred_lead_time
        self.threshold = threshold
        self.threshold_int = threshold
        self.pre_trained = pre_trained
        self.abnormal_limit = abnormal_limit
        self.residual_norm = []


    def fit(self, df_x, y_label, y_response):
        ''' ### Description
        Learn the regression model from the training data, and use the trained model to predict the labels and the repsonse variables for the
            ↪ training data.
        If self.pre_trained is True, it will directly use the pre-trained model.

        ### Parameters
        - df_x: The training features.
        - y_label: The labels in the training dataset.
        - y_response: The response variable in the training dataset.

        ### Returns
        - y_label_pred: The predicted labels using the best regression model learnt from training data.
        - y_response_pred: The predicted response variable using the best regression model learnt from training data.
        '''
        # Train the regression model if not pretrained.
        if not self.pre_trained:
            # Train a regression model first, based on the normal samples.
            # Align indices
            df_x_normal = copy.deepcopy(df_x)
            # Initialize a counter for numbering
            counter = 0
            flag = False
            # Iterate over each row in df
            for i in range(len(y_label)):
                value = y_label.iloc[i]
                if i>0:
                    if value==0 and y_label.iloc[i-1] == 1:
                        flag = True
```

```
                    counter += 1
                if value==1 and y_label.iloc[i-1] == 0:
                    flag = False
                if flag:
                    df_x_normal.at[df_x_normal.index[i], 'test_condition'] += f'_{counter}'

        df_x_normal = df_x_normal[y_label==0]
        y_response_normal = y_response[y_label==0]

        # Train the regression model.
        x_tr, y_temp_tr = prepare_sliding_window(df_x=df_x_normal, y=y_response_normal, window_size=self.window_size,
            ↪ sample_step=self.sample_step, prediction_lead_time=self.pred_lead_time, mdl_type='reg')
        self.reg_mdl = self.reg_mdl.fit(x_tr, y_temp_tr)

    # Calculate and return the predicted labels and response variable for the training data.
    y_label_pred, y_response_pred = self.predict(df_x, y_response)

    return y_label_pred, y_response_pred


def predict(self, df_x_test, y_response_test, complement_truncation=False):
    ''' ### Description
    Predict the labels using the trained regression model and the measured response variable.
    Note that if a fault is predicted, the predicted, not measured response variable will be used to concatenate features
    for predicting the values of next response variable.

    ### Parameters
    - df_x_test: The testing features.
    - y_response_test: The measured response variable.
    - complement_truncation: If True, the truncated points in each sequence before the first sliding window will be complemented. Default is
        ↪ Faulse.

    ### Return
    - y_label_pred: The predicted labels.
    - y_response_pred: The predicted response variable.
    '''
    # Get parameters.
    window_size = self.window_size
    sample_step = self.sample_step
    prediction_lead_time = self.pred_lead_time

    # Handle exception.
    if 'test_condition' not in df_x_test.columns:
        # Add the 'test_condition' column with all values set to 'unspecified'
        df_x_test['test_condition'] = 'unspecified'

    # Get the sequence names.
    sequence_name_list = df_x_test['test_condition'].unique().tolist()

    # Initial values
    y_label_pred = []
    y_response_pred = []

    # Process sequence by sequence.
    for name in tqdm(sequence_name_list):
        # Reset the stored residual errors for normal samples and the threshold value.
        self.residual_norm = []
        self.threshold = self.threshold_int

        # Extract one sequence.
        df_x_test_seq = df_x_test[df_x_test['test_condition']==name]
        y_temp_test_seq = y_response_test[df_x_test['test_condition']==name]
        y_temp_local = copy.deepcopy(y_temp_test_seq)

        # Initial values of the prediction.
        # Length is len - window_size + 1 because we need to use the sliding window to define features.
        y_label_pred_tmp = np.zeros(len(df_x_test_seq)-window_size+1) # Predicted label.
        y_temp_pred_tmp = np.zeros(len(df_x_test_seq)-window_size+1) # Predicted temperature.

        # Making the prediction using a sequential approach.
        for i in range(window_size, len(df_x_test_seq)+1):
            # Get the data up to current moment i-1.
            tmp_df_x = df_x_test_seq.iloc[i-window_size:i, :]
            tmp_y_temp_measure = y_temp_local.iloc[i-window_size:i]

            # Use the same sliding window to generate features.
            feature_x, _ = concatenate_features(df_input=tmp_df_x, y_input=tmp_y_temp_measure, X_window=[], y_window=[],
                    window_size=window_size, sample_step=sample_step, prediction_lead_time=prediction_lead_time, mdl_type='reg')

            # Make prediction.
            tmp_y_label_pred, tmp_y_temp_pred, tmp_residual= self.predict_label_by_reg_base(X=feature_x,
                ↪ y_temp_measure=tmp_y_temp_measure.iloc[-1])

            # Save the prediction at the current moment i.
            y_label_pred_tmp[i-window_size] = tmp_y_label_pred[-1]
            y_temp_pred_tmp[i-window_size] = tmp_y_temp_pred[-1]

            # If we predict a failure, we replace the measure with the predicted temperature.
            # This is to avoid accumulation of errors.
            if tmp_y_label_pred[-1] == 1 and tmp_residual <= self.abnormal_limit:
                y_temp_local.iloc[i-1] = tmp_y_temp_pred[-1]

        # Save the results and proceed to the next sequence.
        if complement_truncation: # If we need to complement, complement with 0.
            len_diff = len(df_x_test_seq) - len(y_label_pred_tmp)
            # If df_x_test_seq is smaller, adjust the length of y_label_pred_tmp
            if len_diff > 0:
                # Create a list of the first element repeated len_diff times
                y_label_pred_tmp = np.concatenate((np.full(len_diff, y_label_pred_tmp[0]), y_label_pred_tmp))
                y_temp_pred_tmp = np.concatenate((np.full(len_diff, y_temp_pred_tmp[0]), y_temp_pred_tmp))
            y_label_pred.extend(y_label_pred_tmp)
            y_response_pred.extend(y_temp_pred_tmp)
```

```
            else:
                y_label_pred.extend(y_label_pred_tmp)
                y_response_pred.extend(y_temp_pred_tmp)

        return y_label_pred, y_response_pred


    def predict_label_by_reg_base(self, X, y_temp_measure):
        ''' ### Description
        Predict the response variable (temperature) using the regression model.
        Then, it saves the residuals of all the normal samples in the current sequence.

        ### Parameters
        - X: The features.
        - y_temp_measure: The temperature measurements.

        ### Return
        - y_label pred: The predicted labels.
        - y_temp_pred: The predicted temperature.
        - residual_error: The residual error.
        '''
        # Predict the temperature
        mdl = self.reg_mdl # Get the regression model.
        y_temp_pred = mdl.predict(X) # Predict the temperature.
        # Calculate the residual
        residual_error = np.array(abs(y_temp_pred - y_temp_measure))

        # Predict the label based on the threshold
        y_label_pred = np.where(residual_error <= self.threshold, 0, 1)

        # Update the residual error database.
        self.residual_norm.extend(residual_error[y_label_pred==0])
        threshold_prop = np.mean(self.residual_norm) + 6*np.std(self.residual_norm)
        # threshold_prop = np.percentile(self.residual_norm, 75) + 1.5*(np.percentile(self.residual_norm, 75)-np.percentile(self.residual_norm,
            ↪ 25))

        if self.threshold < threshold_prop and threshold_prop < 1.5:
            self.threshold = threshold_prop

        return y_label_pred, y_temp_pred, residual_error


    def run_cross_val(self, df_x, y_label, y_response, n_fold=5, single_run_result=True):
        ''' ## Description
        Run a k-fold cross validation based on the testing conditions. Each test sequence is considered as a elementary part in the data.

        ## Parameters:
        - df_X: The dataframe containing the features. Must have a column named "test_condition".
        - y_label: The target variable, i.e., failure.
        - y_response: The response variable associated with the target variable, e.g., temperature.
        - n_fold: The number of folds. Default is 5.
        - single_run_result: Whether to return the single run result. Default is True.

        ## Return
        - perf: A dataframe containing the performance indicators.
        '''

        # Get the unique test conditions.
        test_conditions = df_x['test_condition'].unique().tolist()

        # Define the cross validator.
        kf = KFold(n_splits=n_fold)

        # Set initial values for perf to store the performance of each run.
        perf = np.zeros((n_fold, 4))

        counter = 0
        for train_index, test_index in kf.split(test_conditions):
            # Get the dataset names.
            names_train = [test_conditions[i] for i in train_index]
            names_test = [test_conditions[i] for i in test_index]

            # If not pretrained, train the model.
            if not self.pre_trained:
                df_tr = df_x[df_x['test_condition'].isin(names_train)]
                y_tr = y_label[df_x['test_condition'].isin(names_train)]
                y_response_tr = y_response[df_x['test_condition'].isin(names_train)]

                # Train the model.
                y_tr_pred, y_response_tr_pred = self.fit(df_x=df_tr, y_label=y_tr, y_response=y_response_tr)

            # Extract the training and testing data.
            df_test = df_x[df_x['test_condition'].isin(names_test)]
            y_test = y_label[df_x['test_condition'].isin(names_test)]
            y_response_test = y_response[df_x['test_condition'].isin(names_test)]


            # Predict for the testing data.
            y_pred, y_response_test_pred = self.predict(df_x_test=df_test, y_response_test=y_response_test, complement_truncation=True)

            # Calculate the performance.
            accuracy, precision, recall, f1 = cal_classification_perf(y_test, pd.Series(y_pred))
            perf[counter, :] = np.array([accuracy, precision, recall, f1])

            # Show single run results.
            if single_run_result:
                if self.pre_trained: # Only show the testing performance.
                    # Show the results.
                    fig_1 = plt.figure(figsize = (8,18))
                    axes_test = [fig_1.add_subplot(3, 1, 1), fig_1.add_subplot(3, 1, 2), fig_1.add_subplot(3, 1, 3)]
                    show_reg_result_single_run(y_response_test.reset_index(drop=True), y_response_test_pred, axes_test, 'testing', self.threshold)

```

```
                    fig_2 = plt.figure(figsize = (8,12))
                    ax_test_true = fig_2.add_subplot(2,1,1)
                    ax_test_pred = fig_2.add_subplot(2,1,2)
                    show_clf_result_single_run(y_test, y_pred, ax_test_true, ax_test_pred, suffix='testing')

                    plt.show()
                else:
                    # Truncate the true values for y_test and y_response_test in the same format as the concatenated features.
                    _, y_tr = prepare_sliding_window(df_x=df_tr, y=y_tr, window_size=self.window_size, sample_step=self.sample_step,
                        ↪ prediction_lead_time=self.pred_lead_time, mdl_type='clf')
                    _, y_response_tr = prepare_sliding_window(df_x=df_tr, y=y_response_tr, window_size=self.window_size,
                        ↪ sample_step=self.sample_step, prediction_lead_time=self.pred_lead_time, mdl_type='reg')
                    _, y_response_test = prepare_sliding_window(df_x=df_test, y=y_response_test, window_size=self.window_size,
                        ↪ sample_step=self.sample_step, prediction_lead_time=self.pred_lead_time, mdl_type='reg')

                    # Show the results.
                    show_reg_result(y_tr=y_response_tr, y_test=y_response_test, y_pred_tr=y_response_tr_pred, y_pred=y_response_test_pred,
                        ↪ threshold=self.threshold)
                    show_clf_result(y_tr, y_test, y_tr_pred, y_pred)

            counter += 1

        return pd.DataFrame(data=perf, columns=['Accuracy', 'Precision', 'Recall', 'F1 score'])


def extract_selected_feature_testing(df_data: pd.DataFrame, feature_list: list, motor_idx: int, mdl_type: str):
    ''' ### Description
    Extract the selected features from the dataframe. Used for testing data.
    '''
    feature_list_local = copy.deepcopy(feature_list)
    if mdl_type == 'clf':
        y_name = f'data_motor_{motor_idx}_label'
    elif mdl_type == 'reg':
        y_name = f'data_motor_{motor_idx}_temperature'
    else:
        raise ValueError('mdl_type must be \'clf\' or \'reg\'.')

    if y_name in feature_list_local:
        feature_list_local.remove(y_name)

    feature_list_local.append('test_condition')
    df_x = df_data[feature_list_local]
    y = df_data.loc[:, y_name]

    return df_x, y


def extract_selected_feature(df_data: pd.DataFrame, feature_list: list, motor_idx: int, mdl_type: str):
    ''' ### Description
    Extract the selected features and the response variable from the dataframe.
    '''
    feature_list_local = copy.deepcopy(feature_list)
    if mdl_type == 'clf':
        y_name = f'data_motor_{motor_idx}_label'
    elif mdl_type == 'reg':
        y_name = f'data_motor_{motor_idx}_temperature'
    else:
        raise ValueError('mdl_type must be \'clf\' or \'reg\'.')

    if y_name in feature_list_local:
        feature_list_local.remove(y_name)

    feature_list_local.append('test_condition')
    df_x = df_data[feature_list_local]
    y = df_data.loc[:, y_name]

    return df_x, y


def run_cv_one_motor(motor_idx, df_data, mdl, feature_list, n_fold=5, threshold=3, window_size=1, sample_step=1, prediction_lead_time=1,
    ↪ single_run_result=True, mdl_type='clf'):
    ''' ### Description
    Run cross validation for a given motor and return the performance metrics for each cv run.
    Can be used for both classification and regression models.
    '''
    df_x, y = extract_selected_feature(df_data, feature_list, motor_idx, mdl_type)

    print(f'Model for motor {motor_idx}:')
    df_perf = run_cross_val(mdl, df_x, y, n_fold=n_fold, threshold=threshold, window_size=window_size, sample_step=sample_step,
        ↪ prediction_lead_time=prediction_lead_time, single_run_result=single_run_result, mdl_type=mdl_type)
    print(df_perf)
    print('\n')
    print('Mean performance metric and standard error:')
    for name, metric, error in zip(df_perf.columns, df_perf.mean(), df_perf.std()):
        print(f'{name}: {metric:.4f} +- {error:.4f}')
    print('\n')

    return df_perf


def read_all_test_data_from_path(base_dictionary: str, pre_processing: callable=None, is_plot=True) -> pd.DataFrame:
    ''' ## Description
    Read all the test data from a folder. The folder should contain subfolders for each test. Each subfolder should contain the six CSV files for
        ↪ each motor.
    The test condition in the input will be recorded as a column in the combined dataframe.
    '''
    path_list = os.listdir(base_dictionary)
    path_list = [item for item in path_list if os.path.isdir(os.path.join(base_dictionary, item))]
    path_list = sorted(path_list)

    df_data = pd.DataFrame()
    for tmp_path in path_list:
```

```
        path = base_dictionary + tmp_path
        tmp_df = read_all_csvs_one_test(path, tmp_path, pre_processing)
        df_data = pd.concat([df_data, tmp_df])
        df_data = df_data.reset_index(drop=True)

    df_test_conditions = pd.read_excel(base_dictionary+'Test conditions.xlsx')

    if is_plot:
        for selected_sequence_idx in path_list:
            filtered_df = df_data[df_data['test_condition'] == selected_sequence_idx]
            print('{}: {}\n'.format(selected_sequence_idx, df_test_conditions[df_test_conditions['Test id'] ==
                  ↪ selected_sequence_idx]['Description']))

            fig, axes = plt.subplots(nrows=3, ncols=3, figsize=(15, 10))
            for ax, col in zip(axes.flat, ['data_motor_1_position', 'data_motor_2_position', 'data_motor_3_position',
                'data_motor_1_temperature', 'data_motor_2_temperature', 'data_motor_3_temperature',
                'data_motor_1_voltage', 'data_motor_2_voltage', 'data_motor_3_voltage']):
                label_name = col[:13] + 'label'
                if sum(filtered_df[label_name]==0)>0 or sum(filtered_df[label_name]==1)>0:
                    tmp = filtered_df[filtered_df[label_name]==0]
                    ax.plot(tmp['time'], tmp[col], marker='o', linestyle='None', label=col)
                    tmp = filtered_df[filtered_df[label_name]==1]
                    ax.plot(tmp['time'], tmp[col], marker='x', color='red', linestyle='None', label=col)
                else:
                    ax.plot(filtered_df['time'], filtered_df[col], marker='o', linestyle='None', label=col)
                ax.set_ylabel(col)

            fig, axes = plt.subplots(nrows=3, ncols=3, figsize=(15, 10))
            for ax, col in zip(axes.flat, ['data_motor_4_position', 'data_motor_5_position', 'data_motor_6_position',
                'data_motor_4_temperature', 'data_motor_5_temperature', 'data_motor_6_temperature',
                'data_motor_4_voltage', 'data_motor_5_voltage', 'data_motor_6_voltage']):
                label_name = col[:13] + 'label'
                if sum(filtered_df[label_name]==0)>0 or sum(filtered_df[label_name]==1)>0:
                    tmp = filtered_df[filtered_df[label_name]==0]
                    ax.plot(tmp['time'], tmp[col], marker='o', linestyle='None', label=col)
                    tmp = filtered_df[filtered_df[label_name]==1]
                    ax.plot(tmp['time'], tmp[col], marker='x', color='red', linestyle='None', label=col)
                else:
                    ax.plot(filtered_df['time'], filtered_df[col], marker='o', linestyle='None', label=col)
                ax.set_ylabel(col)

            plt.show()

    return df_data


def read_all_csvs_one_test(folder_path: str, test_id: str = 'unknown', pre_processing: callable = None) -> pd.DataFrame:
    ''' ## Description
    Combine the six CSV files (each for a motor) in a folder into a single DataFrame.
    '''
    csv_files = [file for file in os.listdir(folder_path) if file.endswith('.csv')]
    combined_df = pd.DataFrame()

    for file in csv_files:
        file_path = os.path.join(folder_path, file)
        df = pd.read_csv(file_path)
        df = df.drop(labels=df.columns[0], axis=1)

        if pre_processing:
            pre_processing(df)

        file_name = os.path.splitext(file)[0]
        df = df.add_prefix(f'{file_name}_')
        combined_df = pd.concat([combined_df, df], axis=1)

    df = pd.read_csv(file_path)
    combined_df = pd.concat([df['time'], combined_df], axis=1)

    try:
        time_since_first_row = combined_df['time'] - combined_df['time'].iloc[0]
        combined_df['time'] = time_since_first_row
    except:
        num_rows = combined_df.shape[0]
        time_values = [i * 0.1 for i in range(num_rows)]
        combined_df['time'] = time_values

    combined_df.loc[:, 'test_condition'] = test_id

    label_columns = combined_df.columns[combined_df.columns.str.endswith('_label')]
    non_label_columns = combined_df.columns.difference(label_columns)
    mask = combined_df[non_label_columns].notna().all(axis=1)
    combined_df = combined_df[mask]

    return combined_df


def concatenate_features(df_input, y_input=None, X_window=[], y_window=[], window_size=1, sample_step=1, prediction_lead_time=1, mdl_type='clf'):
    ''' ### Description
    This function takes every sample_step point from a interval window_size, and concatenate the extracted
    features into a new feature list X_window. It extracts the corresponding y in y_window.
    '''
    idx_last_element = len(df_input)-1
    idx_samples = list(reversed(range(idx_last_element, idx_last_element-window_size, -1*sample_step)))
    new_features = df_input.iloc[idx_samples].drop(columns=['test_condition']).values.flatten().tolist()

    if mdl_type == 'reg':
        if prediction_lead_time<=1:
            prediction_lead_time = 1
        if prediction_lead_time<window_size and window_size>1:
            tmp_idx_pred = [x for x in idx_samples if x <= idx_last_element-prediction_lead_time]
            if y_input is not None:
                new_features.extend(y_input.iloc[tmp_idx_pred].values.flatten().tolist())
```

```
    X_window.append(new_features)
    if y_input is not None:
        y_window.append(y_input.iloc[idx_last_element])
        return X_window, y_window
    else:
        return X_window


def prepare_sliding_window(df_x, y=None, sequence_name_list=None, window_size=1, sample_step=1, prediction_lead_time=1, mdl_type='clf'):
    ''' ## Description
    Create a new feature matrix X and corresponding y, by sliding a window of size window_size.
    '''
    X_window = []
    y_window = []

    if sequence_name_list is None:
        sequence_name_list = df_x['test_condition'].unique().tolist()

    def complete_window(df_before_complete):
        if isinstance(df_before_complete, pd.DataFrame):
            is_series = False
            n_size = len(df_before_complete)
        elif isinstance(df_before_complete, pd.Series):
            is_series = True
            n_size = len(df_before_complete)
            df_before_complete = df_before_complete.to_frame(name='Value')
        else:
            raise ValueError("Input must be a pandas DataFrame or Series.")

        if n_size >= window_size:
            return df_before_complete if not is_series else df_before_complete.iloc[0]

        first_row = df_before_complete.iloc[0]
        new_rows = pd.DataFrame([first_row] * (window_size - n_size), columns=df_before_complete.columns)
        df_after_complete = pd.concat([new_rows, df_before_complete], ignore_index=True)

        if is_series:
            df_after_complete = df_after_complete.iloc[:, 0]

        return df_after_complete

    for name in sequence_name_list:
        df_tmp = df_x[df_x['test_condition']==name]
        if y is not None:
            y_tmp = y[df_x['test_condition']==name]

            for i in range(1, window_size):
                df_input = df_tmp.iloc[0:i, :]
                df_input = complete_window(df_input)

                y_input = y_tmp.iloc[0:i]
                y_input = complete_window(y_input)

                X_window, y_window = concatenate_features(df_input=df_input, y_input=y_input,
                    X_window=X_window, y_window=y_window, window_size=window_size, sample_step=sample_step,
                        ↪ prediction_lead_time=prediction_lead_time, mdl_type=mdl_type)

            for i in range(window_size, len(df_tmp)+1):
                X_window, y_window = concatenate_features(df_input=df_tmp.iloc[i-window_size:i, :], y_input=y_tmp.iloc[i-window_size:i],
                    X_window=X_window, y_window=y_window, window_size=window_size, sample_step=sample_step,
                        ↪ prediction_lead_time=prediction_lead_time, mdl_type=mdl_type)
        else:
            for i in range(1, window_size):
                df_input = df_tmp.iloc[0:i, :]
                df_input = complete_window(df_input)
                X_window = concatenate_features(df_input=df_input,
                    X_window=X_window, window_size=window_size, sample_step=sample_step, prediction_lead_time=prediction_lead_time,
                        ↪ mdl_type=mdl_type)

            for i in range(window_size, len(df_tmp)+1):
                X_window = concatenate_features(df_input=df_tmp.iloc[i-window_size:i, :],
                    X_window=X_window, window_size=window_size, sample_step=sample_step, prediction_lead_time=prediction_lead_time,
                        ↪ mdl_type=mdl_type)

    X_window = pd.DataFrame(X_window)

    if y is not None:
        y_window = pd.Series(y_window)
        return X_window, y_window
    else:
        return X_window


def run_cross_val(mdl, df_x, y, n_fold=5, threshold=3, window_size=1, sample_step=1, prediction_lead_time=1, single_run_result=True,
      ↪ mdl_type='reg'):
    ''' ## Description
    Run a k-fold cross validation based on the testing conditions.
    '''
    test_conditions = df_x['test_condition'].unique().tolist()
    kf = KFold(n_splits=n_fold)

    if mdl_type == 'reg':
        perf = np.zeros((n_fold, 3))
    elif mdl_type == 'clf':
        perf = np.zeros((n_fold, 4))
    else:
        TypeError('mdl_type should be either "reg" or "clf".')

    counter = 0
    for train_index, test_index in kf.split(test_conditions):
        names_train = [test_conditions[i] for i in train_index]
```

```
        names_test = [test_conditions[i] for i in test_index]

        X_train, y_train = prepare_sliding_window(df_x, y, names_train, window_size=window_size, sample_step=sample_step,
              ↪ prediction_lead_time=prediction_lead_time, mdl_type=mdl_type)
        X_test, y_test = prepare_sliding_window(df_x, y, names_test, window_size, sample_step=sample_step,
              ↪ prediction_lead_time=prediction_lead_time, mdl_type=mdl_type)

        if mdl_type == 'reg':
            mdl, y_pred_tr, y_pred = run_mdl(mdl, X_train, y_train, X_test)
            perf[counter, :] = np.array([max_error(y_test, y_pred),
            mean_squared_error(y_test, y_pred, squared=False),
            sum(abs(y_pred - y_test)>threshold)/y_test.shape[0]])
            if single_run_result:
                show_reg_result(y_train, y_test, y_pred_tr, y_pred, threshold=threshold)
        elif mdl_type == 'clf':
            mdl, y_pred_tr, y_pred = run_mdl(mdl, X_train, y_train, X_test)
            accuracy, precision, recall, f1 = cal_classification_perf(y_test, y_pred)
            perf[counter, :] = np.array([accuracy, precision, recall, f1])
            if single_run_result:
                show_clf_result(y_train, y_test, y_pred_tr, y_pred)
        else:
            TypeError('mdl_type should be either "reg" or "clf".')

        counter += 1

    if mdl_type == 'reg':
        return pd.DataFrame(data=perf, columns=['Max error', 'RMSE', 'Exceed boundary rate'])
    elif mdl_type == 'clf':
        return pd.DataFrame(data=perf, columns=['Accuracy', 'Precision', 'Recall', 'F1 score'])
    else:
        TypeError('mdl_type should be either "reg" or "clf".')


def cal_classification_perf(y_true, y_pred):
    ''' ### Description
    Accuracy, Precision, Recall and F1 score, with zero-division handling.
    '''
    accuracy = accuracy_score(y_true, y_pred)
    if sum(y_true)==0 and sum(y_pred)==0:
        precision = precision_score(y_true, y_pred, zero_division=1)
        recall = recall_score(y_true, y_pred, zero_division=1)
        f1 = f1_score(y_true, y_pred, zero_division=1)
    else:
        precision = precision_score(y_true, y_pred, zero_division=0)
        recall = recall_score(y_true, y_pred, zero_division=0)
        f1 = f1_score(y_true, y_pred, zero_division=0)

    return accuracy, precision, recall, f1


def run_mdl(mdl, X_tr, y_tr, X_test):
    ''' ## Description
    Fit a model and return predictions on both training and testing sets.
    '''
    mdl = mdl.fit(X_tr, y_tr)
    y_pred_tr = mdl.predict(X_tr)
    y_pred = mdl.predict(X_test)
    return mdl, y_pred_tr, y_pred


def show_reg_result(y_tr, y_test, y_pred_tr, y_pred, threshold=3):
    fig_1 = plt.figure(figsize = (16,18))
    axes_tr = [fig_1.add_subplot(3, 2, 1), fig_1.add_subplot(3, 2, 3), fig_1.add_subplot(3, 2, 5)]
    axes_test = [fig_1.add_subplot(3, 2, 2), fig_1.add_subplot(3, 2, 4), fig_1.add_subplot(3, 2, 6)]
    show_reg_result_single_run(y_tr, y_pred_tr, axes_tr, 'training', threshold)
    show_reg_result_single_run(y_test, y_pred, axes_test, 'testing', threshold)
    plt.show()


def show_reg_result_single_run(y_true, y_pred, axes, suffix='training', threshold=3):
    axes[0].set_xlabel('index of data point', fontsize = 15)
    axes[0].set_ylabel('y', fontsize = 15)
    axes[0].set_title(f'Regression results on the {suffix} dataset', fontsize = 20)
    axes[0].plot(range(len(y_true)), y_true, 'xb', label='Truth')
    axes[0].plot(range(len(y_pred)), y_pred, 'or', label='Prediction')
    axes[0].legend()

    axes[1].set_xlabel('Index of the data points', fontsize = 15)
    axes[1].set_ylabel('Residual error', fontsize = 15)
    axes[1].set_title(f'Residual errors on the {suffix} dataset', fontsize = 20)
    axes[1].plot(y_pred - y_true, 'o')
    axes[1].hlines([threshold, -1*threshold], 0, len(y_true), linestyles='dashed', colors='r')

    axes[2].set_xlabel('Residual errors', fontsize = 15)
    axes[2].set_ylabel('Counts', fontsize = 15)
    axes[2].set_title(f'Distribution of residual errors ({suffix})', fontsize = 20)
    axes[2].hist(y_pred - y_true)
    axes[2].axvline(x=threshold, linestyle='--', color='r')
    axes[2].axvline(x=-1*threshold, linestyle='--', color='r')

    print('\n New run:\n')
    print(f'{suffix} performance, max error is: ' + str(max_error(y_true, y_pred) ))
    print(f'{suffix} performance, mean root square error is: ' + str(mean_squared_error(y_true, y_pred , squared=False)))
    print(f'{suffix} performance, residual error > {threshold}: ' + str(sum(abs(y_true - y_pred)>threshold)/y_true.shape[0]*100) + '%')


def show_clf_result(y_tr, y_test, y_pred_tr, y_pred):
    fig_1 = plt.figure(figsize = (16,12))
    ax_tr_true = fig_1.add_subplot(2,2,1)
    ax_tr_pred = fig_1.add_subplot(2,2,3)
    ax_test_true = fig_1.add_subplot(2,2,2)
    ax_test_pred = fig_1.add_subplot(2,2,4)
```

```
    show_clf_result_single_run(y_true=y_tr, y_pred=y_pred_tr, ax_tr=ax_tr_true, ax_pred=ax_tr_pred, suffix='training')
    show_clf_result_single_run(y_true=y_test, y_pred=y_pred, ax_tr=ax_test_true, ax_pred=ax_test_pred, suffix='testing')
    plt.show()


def show_clf_result_single_run(y_true, y_pred, ax_tr, ax_pred, suffix='training'):
    ax_tr.set_xlabel('index of data point', fontsize = 15)
    ax_tr.set_ylabel('y', fontsize = 15)
    ax_tr.set_title(f'{suffix}: Truth', fontsize = 20)
    ax_tr.plot(range(len(y_true)), y_true, 'xb', label='Truth')
    ax_tr.legend()

    ax_pred.set_xlabel('index of data points', fontsize = 15)
    ax_pred.set_ylabel('y', fontsize = 15)
    ax_pred.set_title(f'{suffix}: Prediction', fontsize = 20)
    ax_pred.plot(range(len(y_pred)), y_pred, 'ro', label='Prediction')
    ax_pred.legend()

    acc, pre, recall, f1 = cal_classification_perf(y_true, y_pred)
    print('\n New run:\n')
    print(f'{suffix} performance, accuracy is: ' + str(acc))
    print(f'{suffix} performance, precision is: ' + str(pre))
    print(f'{suffix} performance, recall: ' + str(recall))
    print(f'{suffix} performance, F1: ' + str(f1))
    print('\n')


if __name__ == '__main__':
    # Demo: pre-train a LinearRegression-based fault detector for motor 6
    # and run it on two test sequences.

    import numpy as np
    import pandas as pd
    from sklearn.pipeline import Pipeline
    from sklearn.preprocessing import StandardScaler
    from sklearn.linear_model import LinearRegression


    def remove_outliers(df: pd.DataFrame):
        df['temperature'] = df['temperature'].where(df['temperature'] <= 100, np.nan)
        df['temperature'] = df['temperature'].where(df['temperature'] >= 0, np.nan)
        df['temperature'] = df['temperature'].ffill()
        df['temperature'] = df['temperature'] - df['temperature'].iloc[0]

        df['voltage'] = df['voltage'].where(df['voltage'] >= 6000, np.nan)
        df['voltage'] = df['voltage'].where(df['voltage'] <= 9000, np.nan)
        df['voltage'] = df['voltage'].ffill()
        df['voltage'] = df['voltage'] - df['voltage'].iloc[0]

        df['position'] = df['position'].where(df['position'] >= 0, np.nan)
        df['position'] = df['position'].where(df['position'] <= 1000, np.nan)
        df['position'] = df['position'].ffill()
        df['position'] = df['position'] - df['position'].iloc[0]


    base_dictionary = 'C:/Users/Zhiguo/OneDrive -
            ↪ CentraleSupelec/Code/Python/digital_twin_robot/projects/maintenance_industry_4_2024/dataset/training_data/'
    df_data = read_all_test_data_from_path(base_dictionary, remove_outliers, is_plot=False)

    normal_test_id = ['20240105_164214',
        '20240105_165300',
        '20240105_165972',
        '20240320_152031',
        '20240320_153841',
        '20240320_155664',
        '20240321_122650',
        '20240325_135213',
        '20240426_141190',
        '20240426_141532',
        '20240426_141602',
        '20240426_141726',
        '20240426_141938',
        '20240426_141980',
        '20240503_164435']

    df_tr = df_data[df_data['test_condition'].isin(normal_test_id)]

    feature_list_all = ['time', 'data_motor_1_position', 'data_motor_1_temperature', 'data_motor_1_voltage',
                    'data_motor_2_position', 'data_motor_2_temperature', 'data_motor_2_voltage',
                    'data_motor_3_position', 'data_motor_3_temperature', 'data_motor_3_voltage',
                    'data_motor_4_position', 'data_motor_4_temperature', 'data_motor_4_voltage',
                    'data_motor_5_position', 'data_motor_5_temperature', 'data_motor_5_voltage',
                    'data_motor_6_position', 'data_motor_6_temperature', 'data_motor_6_voltage']

    x_tr_org, y_temp_tr_org = extract_selected_feature(df_data=df_tr, feature_list=feature_list_all, motor_idx=6, mdl_type='reg')

    window_size = 10
    sample_step = 1
    prediction_lead_time = 1
    threshold = .8
    abnormal_limit = 3

    x_tr, y_temp_tr = prepare_sliding_window(df_x=x_tr_org, y=y_temp_tr_org, window_size=window_size, sample_step=sample_step,
           ↪ prediction_lead_time=prediction_lead_time, mdl_type='reg')

    steps = [
        ('standardizer', StandardScaler()),
        ('regressor', LinearRegression())
    ]
    mdl_linear_regreession = Pipeline(steps)
    mdl = mdl_linear_regreession.fit(x_tr, y_temp_tr)

```

```
    test_id = [
        '20240325_155003',
        '20240425_093699',
    ]
    df_test = df_data[df_data['test_condition'].isin(test_id)]

    detector_reg = FaultDetectReg(reg_mdl=mdl, threshold=threshold, abnormal_limit=abnormal_limit, window_size=window_size,
        ↪ sample_step=sample_step, pred_lead_time=prediction_lead_time)

    _, y_label_test_org = extract_selected_feature(df_data=df_test, feature_list=feature_list_all, motor_idx=6, mdl_type='clf')
    x_test_org, y_temp_test_org = extract_selected_feature(df_data=df_test, feature_list=feature_list_all, motor_idx=6, mdl_type='reg')

    y_label_pred_tmp, y_temp_pred_tmp = detector_reg.predict(df_x_test=x_test_org, y_response_test=y_temp_test_org)

    _, y_label_test = prepare_sliding_window(df_x=x_test_org, y=y_label_test_org, sequence_name_list=test_id, window_size=window_size,
        ↪ sample_step=sample_step, prediction_lead_time=prediction_lead_time, mdl_type='clf')
    _, y_temp_test_seq = prepare_sliding_window(df_x=x_test_org, y=y_temp_test_org, sequence_name_list=test_id, window_size=window_size,
        ↪ sample_step=sample_step, prediction_lead_time=prediction_lead_time, mdl_type='reg')

    show_reg_result(y_tr=y_temp_test_seq, y_test=y_temp_test_seq, y_pred_tr=y_temp_pred_tmp, y_pred=y_temp_pred_tmp,
        ↪ threshold=detector_reg.threshold)
    show_clf_result(y_tr=y_label_test, y_test=y_label_test, y_pred_tr=y_label_pred_tmp, y_pred=y_label_pred_tmp)
```